

Easy Level SQL Queries

1. Find all employees whose names start with "A".

Explanation: You use the **LIKE** operator with the **wildcard character (%)** to match any sequence of zero or more characters after the initial letter 'A'. **Query:**

```
SELECT *  
FROM Employees  
WHERE name LIKE 'A%';
```

2. Count how many employees work in department 'ABC'.

Explanation: You use the **COUNT** aggregate function to count the number of rows that satisfy the specified condition in the **WHERE** clause. **Query:**

```
SELECT COUNT(*)  
FROM Employees  
WHERE department = 'ABC';
```

3. Get employees whose salary is between 10,000 and 50,000.

Explanation: The **BETWEEN** operator provides a concise way to select values within a specified inclusive range. **Query:**

```
SELECT *  
FROM Employees  
WHERE salary BETWEEN 10000 AND 50000;
```

4. Get details of employees whose first name ends with "A" and has exactly 6 letters.

Explanation: The **LIKE** operator is used with the **underscore (_) wildcard**, where each underscore represents exactly one character. Five underscores plus the final 'A' totals six characters. **Query:**

```
SELECT *  
FROM Employees  
WHERE first_name LIKE '_____%A';
```

5. Write a query to fetch duplicate rows from a table.

Explanation: You use the **GROUP BY** clause to group identical rows and the **HAVING** clause to filter those groups where the count is greater than one. **Query:**

```
SELECT column1, column2, COUNT(*)  
FROM YourTable  
GROUP BY column1, column2  
HAVING COUNT(*) > 1;
```

6. Find the employee who has the highest salary.

Explanation: This common technique finds the maximum salary using a subquery and then selects the employee whose salary matches that maximum value. **Query:**

```
SELECT *  
FROM Employees  
WHERE salary = (SELECT MAX(salary) FROM Employees);
```

7. Write a query to get only the first name from a full-name column.

Explanation: Assuming names are separated by a space, you use the **SUBSTRING** and **LOCATE** functions. **LOCATE** finds the position of the first space, and **SUBSTRING** extracts all characters up to that position. **Query:**

```
SELECT SUBSTRING(full_name, 1, LOCATE(' ', full_name) - 1) AS first_name  
FROM Employees;
```

8. Create an empty table with the same structure as another table.

Explanation: You can create an empty copy by running a **CREATE TABLE** command using **LIKE** to copy the structure, including indexes and constraints. **Query:**

```
CREATE TABLE New_Employees LIKE Employees;
```

9. Write a query to increase every employee's salary by 5%.

Explanation: The **UPDATE** command modifies existing records, and the **SET** clause specifies the new calculation. **Query:**

```
UPDATE Employees  
SET salary = salary * 1.05;
```

10. Find all employees hired in the last 8 months.

Explanation: You use the **DATE_SUB** function (or similar date arithmetic depending on the database) to find the date 8 months ago and filter for employees hired after that date. **Query:**

```
SELECT *  
FROM Employees
```

```
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 8 MONTH);
```

11. Find employees who worked in more than one department.

Explanation: Assuming you have a separate EmployeeHistory table, you group the records by employee_id and use the **HAVING** clause to filter groups where the distinct count of departments is greater than one. **Query:**

```
SELECT employee_id  
FROM EmployeeHistory  
GROUP BY employee_id  
HAVING COUNT(DISTINCT department_id) > 1;
```

12. Fetch common records between two tables.

Explanation: You use the **INTERSECT** operator (in databases that support it, like PostgreSQL or Oracle) to return only the rows that exist in both result sets. In MySQL, the same result is achieved using an **INNER JOIN**. **Query (Standard SQL):**

```
SELECT * FROM TableA  
INTERSECT  
SELECT * FROM TableB;  
  
-- Equivalent in MySQL:  
-- SELECT a.* FROM TableA a INNER JOIN TableB b ON a.id = b.id;
```

13. Show users who placed fewer than 3 orders.

Explanation: You use **LEFT JOIN** to include all users, group the results by user_id, and use **COUNT** and **HAVING** to filter out groups where the total number of orders is less than 3. **Query:**

```
SELECT u.user_id, u.name  
FROM Users u  
LEFT JOIN Orders o ON u.user_id = o.user_id  
GROUP BY u.user_id, u.name  
HAVING COUNT(o.order_id) < 3;
```

14. Write a query to get employees with salary >15000 stored across two tables.

Explanation: You use **UNION ALL** to combine the result sets from two separate employee tables, ensuring all employees with the specified salary condition from both tables are returned in a single list. **Query:**

```
SELECT *  
FROM Employees_Table1  
WHERE salary > 15000  
UNION ALL  
SELECT *  
FROM Employees_Table2  
WHERE salary > 15000;
```

JAVA Interview Buddy